Importation des bibliothèques

In [1]:
```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.linear_model import LinearRegression
from sklearn.metrics import  mean_squared_error, r2_score
```

Chargement de dataset

In [2]:
```
data = pd.read_csv("gym_members_exercise_tracking.csv")
data
```

Out[2]:

	Age	Gender	Weight (kg)	Height (m)	Max_BPM	Avg_BPM	Resting_BPM	Session_Duration (hours)	Calories_Burned	Workout_Type	Fat_Percentage
0	56	Male	88.3	1.71	180	157	60	1.69	1313.0	Yoga	12.6
1	46	Female	74.9	1.53	179	151	66	1.30	883.0	HIIT	33.9
2	32	Female	68.1	1.66	167	122	54	1.11	677.0	Cardio	33.4
3	25	Male	53.2	1.70	190	164	56	0.59	532.0	Strength	28.8
4	38	Male	46.1	1.79	188	158	68	0.64	556.0	Strength	29.2
...	...	...	...	...	...	...	...	...	...	...	...
968	24	Male	87.1	1.74	187	158	67	1.57	1364.0	Strength	10.0
969	25	Male	66.6	1.61	184	166	56	1.38	1260.0	Strength	25.0
970	59	Female	60.4	1.76	194	120	53	1.72	929.0	Cardio	18.8
971	32	Male	126.4	1.83	198	146	62	1.10	883.0	HIIT	28.2
972	46	Male	88.7	1.63	166	146	66	0.75	542.0	Strength	28.8

973 rows × 15 columns

Les informations sur dataset

In [3]:
```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 973 entries, 0 to 972
Data columns (total 15 columns):
 #   Column                        Non-Null Count  Dtype  
---  ------                        --------------  -----  
 0   Age                           973 non-null    int64  
 1   Gender                        973 non-null    object 
 2   Weight (kg)                   973 non-null    float64
 3   Height (m)                    973 non-null    float64
 4   Max_BPM                       973 non-null    int64  
 5   Avg_BPM                       973 non-null    int64  
 6   Resting_BPM                   973 non-null    int64  
 7   Session_Duration (hours)      973 non-null    float64
 8   Calories_Burned               973 non-null    float64
 9   Workout_Type                  973 non-null    object 
 10  Fat_Percentage                973 non-null    float64
 11  Water_Intake (liters)         973 non-null    float64
 12  Workout_Frequency (days/week) 973 non-null    int64  
 13  Experience_Level              973 non-null    int64  
 14  BMI                           973 non-null    float64
dtypes: float64(7), int64(6), object(2)
memory usage: 114.2+ KB
```

Vérification des valeurs manquantes

In [4]:
```
#Vérification des valeurs manquantes
data.isnull().sum()
```

Out[4]:
```
Age                              0
Gender                           0
Weight (kg)                      0
Height (m)                       0
Max_BPM                          0
Avg_BPM                          0
Resting_BPM                      0
Session_Duration (hours)         0
Calories_Burned                  0
Workout_Type                     0
Fat_Percentage                   0
Water_Intake (liters)            0
Workout_Frequency (days/week)    0
Experience_Level                 0
BMI                              0
dtype: int64
```

Vérification des valeurs aberrantes

In [5]:
```
#Vérification des valeurs aberrantes
col = ['Age', 'Weight (kg)','Height (m)','Max_BPM','Avg_BPM','Resting_BPM','Session_Duration (hours)',
       'Calories_Burned','Fat_Percentage','Water_Intake (liters)','Workout_Frequency (days/week)','Experience_Level','BMI']
df = data.copy()
for col in col:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1

    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    df = df[(df[col] >= lower_bound) & (df[col] <= upper_bound)]
print("Taille avant :", len(data))
print("Taille après :", len(df))
```

```
Taille avant : 973
Taille après : 931
```

In [6]:
```
#variabe cible
df['Calories_Burned'].value_counts()
```

Out[6]:
```
Calories_Burned
883.0     6
1025.0    6
832.0     5
1046.0    4
1150.0    4
         ..
862.0     1
723.0     1
1127.0    1
1113.0    1
812.0     1
Name: count, Length: 598, dtype: int64
```

Séparartion des données

In [7]:
```
X = df.drop(columns=['Calories_Burned'])
y = df['Calories_Burned']
```

In [8]:
```
y = y.values.reshape(-1, 1)
```

In [9]:
```
print(f"Dataset shape: {X.shape}")
print(f"Target shape: {y.shape}")
```

```
Dataset shape: (931, 14)
Target shape: (931, 1)
```

Identification des colonnes catégorielles et numériques

In [10]:
```
#Identification des colonnes catégorielles et numériques
cat_cols = X.select_dtypes(include=['object', 'category']).columns
num_cols = X.select_dtypes(include=['int64', 'float64']).columns
```

Encodage des colonnes catégorielles

In [11]:
```
#Encodage des colonnes catégorielles
le = LabelEncoder()
for col in cat_cols:
   X[col] = le.fit_transform(X[col])
```

Standardisation des colonnes numériques

In [12]:
```
#Standardisation des colonnes numériques
scaler = StandardScaler()
X[num_cols] = scaler.fit_transform(X[num_cols])
```

In [166…
```
X["bias"] = 1
```

Division des données

In [13]:
```
# Division des données
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

In [14]:
```
print(X_train.shape)
print(X_test.shape)
print(y_train.shape)
print(y_test.shape)
```

```
(744, 14)
(187, 14)
(744, 1)
(187, 1)
```

Création du modèle

In [169…
```
# Initialisation de theta
theta = np.zeros((X_train.shape[1], 1))
```

In [170…
```
# Modèle
def model(X, theta):
    return X.dot(theta)

# Fonction coût
```

```
def fonction_cout(X, y, theta):
    m = len(y)
    predictions = model(X, theta)
    return (1/(2*m)) * np.sum((predictions - y)**2)
```

In [171…
```
# Descente de gradient
def descente_gradient(X, y, theta, learning_rate=0.01, n_iterations=1000):
    m = len(y)
    cout_h = []

    for i in range(n_iterations):
        predictions = modele(X, theta)
        derive = (1/m) * X.T.dot(predictions - y)
        theta = theta - learning_rate * derive

        cout = fonction_cout(X, y, theta)
        cout_h.append(cout)

    return theta, cout_h
```

Entraînement du modèle

In [172…
```
# Entraînement du modèle
theta_opt, historique = descente_gradient(X_train.values, y_train, theta)
```

In [173…
```
# Prédictions
y_pred = model(X_test.values, theta_opt)
```

Evaluation du modèle

In [174…
```
# MSE
mse = mean_squared_error(y_test, y_pred)
print(f"MSE final: {mse:.3f}")
```

```
MSE final: 3961.087
```

In [175…
```
print("R2:",  r2_score(y_test, y_pred))
```

```
R2: 0.9455110550984301
```

In [16]:
```
# création du modèle
model = LinearRegression()

# entrainement du modèle
model.fit(X_train, y_train)

# faire des prédictions
predictions = model.predict(X_test)

# Evaluation
mse = mean_squared_error(y_test, predictions)
print("MSE:", mse)
print("R2:",  r2_score(y_test, predictions))
```

```
MSE: 1400.984932794746
R2: 0.9807279713282295
```

Enregistrement du modèle

In [18]:
```
import joblib
joblib.dump(model, "RegL_model.pkl")
print("Modele sauvegarde sous le nom 'RegL_model.pkl'")
```

```
Modele sauvegarde sous le nom 'RegL_model.pkl'
```